

Realiza las siguientes actividades propuestas, y elabora con ellas un informe y una pequeña presentación.

El informe incorporará capturas de pantalla de las acciones realizadas, así como respuesta a aquellas cuestiones que se planteen en los apartados, y durante la sesión de presentaciones. Se incluirá en el portafolios que entregaremos al final de las prácticas.

La presentación, orientativamente, durará 10 minutos, y debe contener los problemas y resultados encontrados durante el trabajo realizado en la primera sesión de prácticas.

PULL REQUESTS, ISSUES Y PROYECTOS

GitHub, como tecnología de control de versiones, ofrece dos tecnologías potentes para el desarrollo en colaboración:

- Issues: la pestaña de issues de un proyecto permite a personas que no pertenecen al repositorio dejar comentarios, feedback, publicar bugs, etc. Un ejemplo de issue, en el repositorio de LLVM:

<https://github.com/llvm/llvm-project/issues/179455>

Se puede ver que una persona puede notificar, en formato de foro, un bug que ha encontrado, y que potencialmente podría recibir respuesta ó solicitudes de contexto del equipo desarrollador.

- Forks/Pull Requests: a través de estos dos verbos de GitHub, una persona puede, a diferencia de con las issues, proponer la incorporación de cambios que ya haya realizado personalmente. Un ejemplo de pull request, en el repositorio de nginx:

<https://github.com/nginx/nginx/pull/1117>

Se ve cómo el usuario propone una ampliación del repositorio, añadiendo una feature, y GitHub permite al equipo desarrollador del proyecto auditar la propuesta de inclusión, y aprobarla ó denegarla.

Realiza las siguientes actividades:

1. Observa el proyecto:

<https://github.com/javillo-ur/PullRequestsExample>

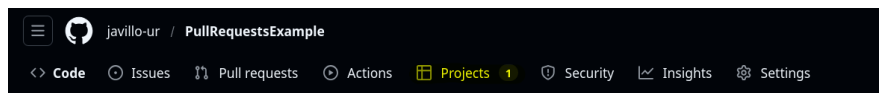
Tiene clases que se corresponden con nuestras CUASIs, pero no las vuestras. Propón que incluyamos una clase con tu CUASI. Para ello, realiza los siguientes pasos:

- Crea un fork del proyecto. Con esto tendrás, en tu propia cuenta, una copia enlazada de nuestro proyecto a tu nombre
- Edita desde algún IDE tu repositorio para hacer los cambios pertinentes, y haz push a tu propia copia
- Desde tu copia del repositorio, propón una pull request para que incluyamos tu CUASI en el repositorio original

¿Qué opciones se te ofrecen a la hora de añadir detalles a la pull request?

¿Qué ventajas tiene una forma de trabajo basada en pull requests, contra un equipo que comparta el repositorio? ¿En qué situaciones tiene ó deja de tener sentido?

2. Crea un nuevo repositorio, y crea sobre él un nuevo proyecto en la pestaña de proyectos:



En las plantillas de proyectos, haz que sea de tipo Kanban. Cámbiale el nombre, y si no tiene una vista de tablón, crea una nueva vista de tablón.

Esto nos ha creado un tablón Kanban virtual, al estilo de metodologías ágiles, que puede compartir el equipo de desarrollo.

Entra ahora a la pestaña de issues de tu proyecto. Crea varias issues, observando que las puedes añadir como items al tablero Kanban. ¿Qué otras configuraciones puedes cambiar en las issues?

Usar las issues como elementos del tablón, permite asignar a personas del equipo desarrollador cada issue, así que se puede mantener seguimiento de quién tiene asignadas qué tareas. Además, permite añadir las sugerencias de la base de usuarios rápidamente al planning del equipo desarrollador.

¿Para qué nivel de granularidad crees que tiene más sentido usar esta herramienta (un proyecto por repositorio, un proyecto por versión del repositorio, etc)? ¿Qué clase de ventajas da tener un tablón a nivel de coordinación?

Nota: usaremos esta feature para organizar el trabajo en grupo de la asignatura.

Incluye capturas de pantalla de los pasos que hayas realizado en el transcurso de la actividad.

GITHUB PAGES Y GENERADORES DE WEBS ESTÁTICAS

Para el siguiente apartado, trabajaremos con tres tecnologías distintas. **Sólo debes usar una:**

- **Hugo**, si la última cifra de tu DNI es 0, 1 ó 2
- **Pelican**, si la última cifra de tu DNI es 3, 4 ó 5
- **Jekyll**, si la última cifra de tu DNI es 6, 7, 8 ó 9

Estas tecnologías son generadores de páginas web estáticas. Permiten crear ficheros de texto plano que automáticamente el sistema convierte en páginas HTML+CSS, con un formato de curriculum vitae y blog. Mucha gente los usa, por ejemplo, como plantilla para una página web personal.

Están basadas cada una en un lenguaje distinto:

- Hugo, basada en Go
- Pelican, basada en Python
- Jekyll, basada en Ruby

¿Qué conoces de estos lenguajes? ¿Puedes anticipar algo sobre cómo van a funcionar las herramientas (velocidad, flexibilidad, modularidad)?

Instalación

De cara a instalar cada herramienta:

- Se recomienda instalar **Hugo** usando WinGet, tal cuál se detalla en la página de la herramienta
- La forma más sencilla de usar Python y sus paquetes en los ordenadores de laboratorio es a través de Anaconda. Se puede encontrar en las aplicaciones virtualizadas. Python tiene un gestor de paquetes para sus librerías (pip) que permite instalar **Pelican**, tal cuál se detalla en la página de instalación
- Para instalar **Jekyll**, sí que hace falta realizar una instalación de Ruby. La mejor forma de hacerlo para nuestros propósitos es seguir la guía en la página del propio Jekyll

Creación de la página

Para las tres tecnologías, realizaremos lo siguiente:

- Crearemos un proyecto base (para cada tecnología existe su propio comando de consola). Documenta este proceso

- Iniciaremos un repositorio de git, que subiremos a un repositorio de nuestro GitHub como remote
- Lanzaremos el proyecto base desde consola en el ordenador local, para ver cómo se ve la página por defecto

Las tres herramientas crean páginas web en base a documentos de Markdown (extension .md). Explora los archivos que han creado. No vamos a tener que escribir código en ninguno de los tres lenguajes, pero trata de interpretar qué sintaxis usa en las páginas MD (¿Cómo usa argumentos? ¿Cómo se configura? ¿Cómo se cambia el título de los posts del blog?).

Workflows de GitHub

GitHub ofrece la posibilidad de desplegar GitHub Pages, es decir, que GitHub aloje por ti una página web estática (¿Qué significa que una web sea estática?) en uno de sus dominios. Es una forma habitual en la industria de tener una página web personal en la que se pueda ver tu curriculum vitae, a la vez que barata.

Ahora bien, lo que GitHub tiene es el código fuente de la página, no el HTML. Sin embargo, nos ofrece acciones y workflows de GitHub, que permiten demandarle al servidor que compile la página antes de publicarla en abierto.

Ve a la configuración del proyecto, y asegúrate de que su visibilidad es pública (se pueden crear GitHub Pages con repositorios privados, pero esta opción es de pago). Una vez hecho esto, también en la configuración del proyecto, ve al apartado de Pages.

En Pages, GitHub ofrece dos opciones, desplegar desde una branch (la opción por defecto), ó desde GitHub Actions. Selecciona GitHub Actions, y debería reconocer que puede aplicar la acción por defecto para la tecnología que hayas usado. GitHub tiene un marketplace de acciones, que ofrece muchas opciones para cómputo en la nube sin tener que programarlas.

Guarda la acción por defecto en tu repositorio. ¿Dónde se ha guardado? Abre el archivo del workflow, y trata de interpretarlo. (¿Cuándo se ejecutará la acción? ¿Qué hará?).

Despliegue

Altera los Markdown de la página con tu editor favorito, y haz un push. Una vez que la acción se acabe de ejecutar (entra a la pestaña de GitHub Actions) para ver el proceso de ejecución. Vuelve a la pestaña de Pages de la configuración. ¿Es pública la web? ¿En qué URL? ¿Puedes cambiar la dirección en la que se publica? Deja la página a tu gusto.

RESULTADOS FINALES Y LECCIONES APRENDIDAS

¿Qué cosas has aprendido ó descubierto durante la realización de la práctica que no conocieses previamente?